

Infinity OS — Cognitive Invariance Layer v1.0

Formal Architecture Specification

Infinity Invariant Engine · Substrate Governance · Epoch Lifecycle · Conformance

System

Infinity OS / Cognitive Invariance Layer + Invariant Engine

Document Type

Formal Architecture Specification (FAS)

Version

v1.0

Status

DRAFT-STABLE

Date

June 2026

Companion Document

Infinity OS — CIL v1.0 Engineering Specification

Audience

Senior Architects, Governance Engineers, System Designers

Classification

Internal — Architecture Governance

NORMATIVE NOTICE

This document constitutes a normative architecture specification. All language marked **MUST**, **MUST NOT**, and **SHALL** expresses binding conformance requirements. All six invariants defined in Section 5 are non-negotiable for CIL-conformant systems. Deviations require a formal governance waiver under the Infinity Consensus Contract.

Table of Contents

1. Purpose & Scope

1.1 Purpose

1.2 Scope

2. Terminology

3. Architecture Overview

3.1 System Model

3.2 Design Principles

3.3 Substrate Support Matrix

4. Layer Definitions & Boundary Rules

4.1 Cognitive Layer (CL)

4.2 Meta-Substrate Layer (MSL)

4.3 Substrate Layer (SL)

4.4 Boundary Violation Classification

5. Core Invariants (Normative)

INV-1 Deterministic Cognition

INV-2 Layer Separation

INV-3 Epoch Semantics

INV-4 Governance Supremacy

INV-5 Auditability

INV-6 Physics Threat Handling

6. Governance Architecture

6.1 Infinity Design Law — Law Hierarchy

6.2 Infinity Consensus Contract

6.3 GovernanceTicket — Canonical Governance Envelope

7. Infinity Invariant Engine (IIE) — Component Architecture

7.1 Components

7.2 IIE Integration Points

8. System Flows

Flow A — Epoch Proposal & Pre-Validation

Flow B — Execution Guarding

Flow C — Post-Execution Verification

Flow D — Drift & PhysicsThreat Handling

End-to-End Canonical Example

9. Conformance Requirements

10. System Properties & Guarantees

11. Appendix — Invariant Reference Table

1. Purpose & Scope

1.1 Purpose

This document defines the formal architecture of the **Cognitive Invariance Layer (CIL) v1.0** for **Infinity OS**. The CIL specifies substrate-independent semantics for cognitive runtimes such that system behavior remains deterministic, auditable, and governed across classical, quantum, lattice, and hybrid execution substrates.

CIL v1.0 is composed of two inseparable, co-specified components:

- **CIL — Cognitive Invariance Layer:** The normative invariant model. Defines what **MUST** always be true of any conformant cognitive runtime, independent of substrate, implementation language, or deployment topology.
- **IIE — Infinity Invariant Engine:** The enforcement engine. The cross-cutting runtime component that ensures CIL invariants hold at all times, intercepts violations, routes governance signals, and maintains the tamper-evident audit ledger.

Together, CIL and IIE establish the foundational contract that any Infinity OS cognitive runtime **MUST** satisfy in order to claim conformance. Neither component is optional, and neither is sufficient alone.

1.2 Scope

In scope for this specification:

- Cognitive layer semantics and the normative invariant model
- Epoch lifecycle definition and state transition rules
- IIE component architecture, responsibilities, and enforcement flows
- Governance integration architecture and the Infinity Consensus Contract
- Meta-Substrate Layer (MSL) boundary contract and interface definitions
- Formal conformance requirements (C-01 through C-22)
- System properties and formal guarantees (P-1 through P-6)

Out of scope for this specification:

- Concrete substrate implementations (CPU, GPU, QPU, lattice hardware specifics)
- Specific consensus algorithm implementations
- Operator UX, dashboard design, and tooling interfaces
- Network transport protocols and inter-node communication

- Application-layer cognitive algorithms and model architectures

Companion Document: Concrete implementation guidance, substrate-specific bindings, API schemas, and engineering-level detail are specified in the companion document: *Infinity OS — CIL v1.0 Engineering Specification*. This FAS establishes the normative architecture contract; the Engineering Specification provides implementation realizations.

2. Terminology

The following terms are used with precise, defined meanings throughout this specification. All definitions are normative. Terms are rendered in **bold** on first occurrence in each section.

Term	Definition
Cognitive Layer (CL)	The logical layer where cognition, planning, and reasoning occur. Contains no substrate-specific logic. All CL state changes MUST be expressed as typed epoch transitions. CL interacts with SL exclusively through MSL.
Substrate Layer (SL)	The concrete execution environment — classical CPU/GPU, quantum QPU, lattice, or hybrid. Contains no governance logic and no cognitive decision logic. All SL interaction with upper layers MUST pass through MSL.
Meta-Substrate Layer (MSL)	The strict abstraction boundary between CL and SL. The only legal interface between the two layers. Zero cognition and zero governance occur inside MSL. MSL translates epoch workloads downward and normalizes physics events upward.
Epoch	The minimal, atomic unit of committed CL

Term	Definition
	state transition. All CL state changes MUST occur as typed epochs. An epoch is either fully committed or fully absent from canonical state — no partial epoch is observable.
Invariant	A property that MUST always hold for a CIL-conformant system. Invariants are defined normatively in Section 5. Enforced at runtime by IIE. A system is CIL-conformant only if all six invariants hold at all times.
IIE	Infinity Invariant Engine. The cross-cutting enforcement component that validates, guards, and verifies all CL state transitions and governance interactions. IIE is not a layer — it is a runtime process spanning all layer transitions.
PhysicsThreat	A classified substrate-level anomaly — including decoherence, entanglement poisoning, lattice drift, and timing anomaly — that has been normalized by MSL and classified by IIE's Drift & Threat Detector. MUST be governed; MUST NOT be silently ignored.
Violation	The canonical error object representing any breach of an invariant, Design Law, or governance constraint. Violations are surfaced by IIE components and wrapped in GovernanceTickets for governance routing.
GovernanceTicket	The canonical normalized envelope for all governance-relevant signals — violations, physics threats, and composite events. All signals entering the governance layer MUST be expressed as GovernanceTickets.
Design Law	The four-level normative law hierarchy (L0 Constitutional through L3 Guideline) that governs all Infinity OS system behavior. Defined in Section 6.1. L0 laws are non-overrideable by construction.
SAI Profile	Substrate Abstraction Interface profile. The declared capabilities, constraints, and determinism contract of a specific substrate instance, as returned by <code>MSL.DescribeSubstrate()</code> . Used by IIE during pre-execution validation to confirm substrate compatibility.

3. Architecture Overview

3.1 System Model

CIL v1.0 defines a **three-layer execution model** with one cross-cutting enforcer. The three layers are strictly separated by defined boundary contracts. The **IIE** is not a layer — it is a cross-cutting process that enforces invariants across all layer transitions.

▲ MSL boundary — sole legal interface ▼
Meta-Substrate Layer (MSL) Strict RPC boundary · Zero cognition · Zero governance · Capability declaration · Normalization
▲ SL isolation boundary ▼
Substrate Layer (SL) Classical · Quantum · Lattice · Hybrid · Raw PhysicsEvent emission
↕ <i>enforced across all transitions by</i> ↕
Infinity Invariant Engine (IIE) Cross-cutting · Pre / Runtime / Post enforcement · Governance bridge · Threat detection · Audit ledger

3.2 Design Principles

The following six design principles are foundational to CIL v1.0. They are not guidelines — they are architectural axioms that shape every conformance requirement in this document.

1. **Substrate Independence.** CL behavior is defined independently of any specific SL. Substrate nondeterminism — including quantum probabilism, lattice drift, and timing variance — **MUST NOT** leak into CL. All nondeterminism **MUST** be absorbed at or below MSL.

2. **Governance Supremacy.** Governance constraints supersede all runtime optimizations, substrate capabilities, and performance heuristics. No execution path exists that may deliver a CL state change while bypassing IIE enforcement or active governance constraints.
3. **Epoch Atomicity.** No CL state change exists outside of a typed epoch transition. Ad-hoc mutation is forbidden by construction. An epoch is either fully committed or fully absent from the canonical state — partial epoch state is never observable.
4. **Auditability by Default.** All state transitions, governance decisions, and physics events are logged in a tamper-evident, replayable ledger. Auditability is a first-class architectural requirement, not a post-hoc observability feature.
5. **Strict Layer Separation.** Each layer has a single defined responsibility. Cross-layer coupling — cognitive logic in MSL, substrate logic in CL, governance logic in SL — is a conformance violation detected and halted by IIE.
6. **Governed Physics.** Substrate anomalies are first-class governance events. A **PhysicsThreat** is not an implementation detail to be absorbed silently — it **MUST** be detected, classified, and governed through the defined IIE and governance pathway.

3.3 Substrate Support Matrix

Substrate Type	Classification	CIL Support Status	Known PhysicsThreat Classes
Classical (CPU / GPU)	SL-CLASSICAL	Full	TIMING_ANOMALY
Quantum (QPU)	SL-QUANTUM	Full	COHERENCE_COLLAPSE, ENTANGLEMENT_POISONING
Lattice	SL-LATTICE	Full	LATTICE_DRIFT
Hybrid	SL-HYBRID	Full	All of the above +

Substrate Type	Classification	CIL Support Status	Known PhysicsThreat Classes
			UNKNOWN

All substrate types listed above are first-class citizens under CIL v1.0. **UNKNOWN** PhysicsThreat class is reserved for hybrid substrates encountering anomaly patterns that do not match any defined category; such events **MUST** be escalated to governance at CRITICAL severity by default.

4. Layer Definitions & Boundary Rules

4.1 Cognitive Layer (CL)

The **Cognitive Layer (CL)** is the logical layer where all cognition, planning, reasoning, and agent orchestration occur. It is the only layer that may contain cognitive decision logic.

Responsibilities:

- Cognition, planning, reasoning, and agent orchestration
- Epoch construction and proposal via `IIE.ProposeEpoch()`
- Policy evaluation and governance interaction — exclusively through IIE
- Consumption of governance decisions delivered by IIE

Hard Boundary Rules — CL MUST NOT:

- Contain any substrate-specific logic — no quantum circuits, qubit operations, GPU kernel calls, or memory addressing schemes

- Call SL directly — all SL interaction **MUST** route through MSL only; direct CL→SL calls are a FATAL conformance violation
- Express any state change outside of a typed epoch transition — ad-hoc mutation is forbidden by construction
- Implement or evaluate governance constraints — all governance interaction occurs via IIE

4.2 Meta-Substrate Layer (MSL)

The **Meta-Substrate Layer (MSL)** is the strict abstraction boundary between CL and SL. It is the sole legal interface between these two layers. MSL is a translation and normalization layer — it contains neither cognitive logic nor governance logic.

Responsibilities:

- Declare substrate capabilities and constraints via **SubstrateProfile** (SAI Profile)
- Accept `EpochWorkload` from CL and translate to SL-executable form
- Execute substrate workload under the determinism contract declared in `SubstrateProfile`
- Translate raw SL `PhysicsEvent` signals into normalized **PhysicsThreat** for IIE
- Stream substrate telemetry to IIE Drift & Threat Detector (DTD)

Formal MSL Interface:

```
MSL.DescribeSubstrate()           → SubstrateProfile
MSL.ExecuteEpochWorkload(EpochWorkload) → EpochWorkloadResult
MSL.StreamMetrics(Subscription)    → stream
<
```

SubstrateMetric

>

MSL.ReportPhysicsEvent(PhysicsEvent) [SL → MSL → IIE normalization path]

Hard Boundary Rules — MSL MUST:

- Contain **ZERO** cognitive logic — no reasoning, no planning, no policy evaluation
- Contain **ZERO** governance logic — no law evaluation, no constraint enforcement, no invariant checking
- Be the **ONLY** interface between CL and SL — no side channels, shared memory segments, or out-of-band signals are permitted
- Normalize all SL anomalies into the defined `PhysicsThreat` schema before passing upward to IIE
- **NOT** modify CL state — MSL is stateless with respect to the CL canonical state

4.3 Substrate Layer (SL)

The **Substrate Layer (SL)** is the concrete execution environment. It provides raw computational resources and emits raw physics events on anomaly detection. SL contains no cognitive or governance logic.

Responsibilities:

- Concrete execution of epoch workloads received from MSL
- Emission of raw `PhysicsEvent` signals on anomaly detection (decoherence, drift, etc.)
- Reporting resource usage metrics to MSL telemetry stream

- Adherence to the determinism contract declared in its SubstrateProfile

Hard Boundary Rules — SL MUST NOT:

- Contain cognitive decision logic of any kind
- Contain governance constraints or invariant evaluation logic
- Interact upward with CL directly — all upward interaction **MUST** pass through MSL
- Interpret the intent of CL epoch workloads — workloads are treated as opaque execution units

4.4 Boundary Violation Classification

The following table defines the canonical boundary violations, their severity classification, and the required IIE response. All violations generate a **GovernanceTicket**.

Violation Type	Example	Severity	IIE Response
CL → SL direct call	CL invokes QPU API directly, bypassing MSL	FATAL	HALT + GovernanceTicket (VIOLATION)
SL → CL direct signal	SL modifies CL state directly, bypassing MSL	FATAL	HALT + GovernanceTicket (VIOLATION)
Governance logic in MSL	MSL evaluates an invariant or law constraint	ERROR	BLOCK + ESCALATE to governance
Substrate logic in CL	CL contains qubit operations or GPU kernel calls	FATAL	HALT + GovernanceTicket (VIOLATION)
MSL state mutation	MSL modifies or writes to CL canonical state	FATAL	HALT + GovernanceTicket (VIOLATION)

5. Core Invariants (Normative)

NORMATIVE STATEMENT

The six invariants defined in this section are the normative core of CIL v1.0. A system is CIL-conformant **if and only if** all six invariants hold at all times. Invariant violations are not recoverable conditions — they require formal governance intervention via a **GovernanceTicket** before execution may resume. No invariant may be disabled, waived, or overridden without an authorized governance decision under the Infinity Consensus Contract.

INV-1

—

Deterministic Cognition

Formal Statement	For any initial CL state <i>S</i> and input <i>I</i> , the resulting state <i>S'</i> and outputs <i>O</i> MUST be identical across all conformant substrates. Substrate nondeterminism — including quantum probabilism, lattice drift, and timing variance — MUST be absorbed at or below MSL. It MUST NOT propagate into CL.
Constraint	No substrate characteristic may produce divergent CL outputs given identical inputs. Determinism is a property of CL semantics, not of substrate execution mechanics.
Enforcement	PEV pre-checks determinism constraints before execution is authorized. PXV post-checks output hash consistency after execution completes.

Formal Statement	For any initial CL state S and input I , the resulting state S' and outputs O MUST be identical across all conformant substrates. Substrate nondeterminism — including quantum probabilism, lattice drift, and timing variance — MUST be absorbed at or below MSL. It MUST NOT propagate into CL.
Check Points	PRE-EXECUTION (PEV) + POST-EXECUTION (PXV)
Severity	FATAL
IIE Response	HALT + GovernanceTicket(COMPOSITE) — physics anomaly and invariant breach co-reported when caused by substrate nondeterminism.

INV-2
—
Layer Separation

Formal Statement	(a) CL MUST NOT contain substrate-specific logic. (b) SL MUST NOT contain cognitive or governance decision logic. (c) All CL↔SL interaction MUST pass exclusively through MSL — no direct channels, side-band signals, or shared state are permitted.
Constraint	Layer responsibility boundaries are enforced structurally, not by convention. Cross-layer coupling of any form is a conformance violation.
Enforcement	RG monitors all cross-layer calls at runtime and blocks any illegal channels or direct CL↔SL communication.
Check Points	PRE-EXECUTION (PEV) + RUNTIME (RG)

Formal Statement	(a) CL MUST NOT contain substrate-specific logic. (b) SL MUST NOT contain cognitive or governance decision logic. (c) All CL↔SL interaction MUST pass exclusively through MSL — no direct channels, side-band signals, or shared state are permitted.
Severity	FATAL
IIE Response	HALT + GovernanceTicket(VIOLATION)

INV-3
—
Epoch Semantics

Formal Statement	All CL state changes MUST occur as typed epoch transitions. Epoch transitions MUST satisfy all four of the following properties: (a) Typed: One of CREATE MUTATE ROLLBACK GOVERNANCE CHECKPOINT. (b) Deterministic: Given identical initial state <i>S</i> and input <i>I</i> , the resulting state <i>S'</i> and outputs <i>O</i> are identical. (c) Idempotent: Replay from the same checkpoint produces identical results. (d) Governed: Every epoch MUST be bound to a governance context (law context, proposal <i>I</i>).
Constraint	No "out-of-epoch" mutation is permitted under any circumstance. Ad-hoc state mutation — whether for performance, recovery, or convenience — is a conformance violation.
Enforcement	PEV validates epoch type and legal transition before authorization. RG enforces epoch semantics during execution. PXV confirms post-state consistency and idempotency.

Formal Statement	All CL state changes MUST occur as typed epoch transitions. Epoch transitions MUST satisfy all four of the following properties: (a) Typed: One of <code>CREATE</code> <code>MUTATE</code> <code>ROLLBACK</code> <code>GOVERNANCE</code> <code>CHECKPOINT</code> . (b) Deterministic : Given identical initial state <i>S</i> and input <i>I</i> , the resulting state <i>S'</i> and outputs <i>O</i> are identical. (c) Idempotent : Replay from the same checkpoint produces identical results. (d) Governed : Every epoch MUST be bound to a governance context (<code>LAW_CONTEXT</code> , <code>PROPOSAL_ID</code>).
Check Points	PRE-EXECUTION (PEV) + RUNTIME (RG) + POST-EXECUTION (PXV)
Severity	FATAL (unlawful mutation) ERROR (malformed epoch)
IIE Response	REJECT epoch + GovernanceTicket(VIOLATION)

INV-4	
—	
Governance Supremacy	

Formal Statement	Governance decisions — including laws, constraints, vetoes, and waivers — MUST remain valid and enforced regardless of substrate behavior, performance concerns, or runtime heuristics. Substrate-level anomalies MUST NOT bypass or override governance.
Constraint	No component — CL, MSL, SL, or IIE itself — may override an active governance decision without a subsequent authorized governance event. Performance optimization is explicitly not a valid justification for governance bypass.

Formal Statement	Governance decisions — including laws, constraints, vetoes, and waivers — MUST remain valid and enforced regardless of substrate behavior, performance concerns, or runtime heuristics. Substrate-level anomalies MUST NOT bypass or override governance.
Enforcement	GB ensures all active governance decisions are validated and applied by IIE before execution resumes after any HALT or BLOCK event.
Check Points	ALL (PRE + RUNTIME + POST)
Severity	FATAL
IIE Response	HALT + GovernanceTicket(VIOLATION, CRITICAL)

INV-5
—
Auditability

Formal Statement	All of the following MUST be logged in a tamper-evident, replayable audit ledger: (a) All epoch transitions — in all states: PROPOSED, AUTHORIZED, EXECUTING, COMMITTED, REJECTED, ROLLED_BACK. (b) All governance decisions. (c) All PhysicsThreat events (at all severity levels). (d) All Violation events. (e) All IIE enforcement actions.
Constraint	Audit logs MUST be append-only. No entry may be modified or deleted post-write under any operational or maintenance procedure. Retroactive modification is a conformance violation regardless of operator authorization level.
Enforcement	IIE writes to the ledger at each transition

Formal Statement	All of the following MUST be logged in a tamper-evident, replayable audit ledger: (a) All epoch transitions — in all states: PROPOSED , AUTHORIZED , EXECUTING , COMMITTED , REJECTED , ROLLED_BACK . (b) All governance decisions. (c) All PhysicsThreat events (at all severity levels). (d) All Violation events. (e) All IIE enforcement actions .
	point. PXV verifies log completeness and tamper-evidence post-execution before permitting epoch commit.
Check Points	POST-EXECUTION (PXV) + continuous write at each transition
Severity	FATAL (missing required entry) ERROR (incomplete entry)
IIE Response	Block epoch commit until audit entry is confirmed written and verified tamper-evident.

INV-6
—
Physics Threat Handling

Formal Statement	All substrate physics anomalies MUST be: (a) Detected — by DTD continuously monitoring substrate telemetry via MSL. (b) Classified — assigned a category (QUANTUM COLLAPSE , ENTANGLEMENT POISONING , ARTIFICIAL BREIF , EMISSIVE ANOMALY , DEFCON) and severity (INFO , WARNING , CRITICAL). (c) Governed — routed through the defined IIE → GB → governance pathway.
Constraint	CL MUST NOT silently continue after a classified CRITICAL PhysicsThreat . Emergency actions taken before

Formal Statement	All substrate physics anomalies MUST be: (a) Detected — by DTD continuously monitoring substrate telemetry via MSL. (b) Classified — assigned a category (<code>COHERENCE_COLLAPSE</code> , <code>ENTANGLEMENT_POISONING</code> , <code>LATTICE_DRIFT</code> , <code>TIMING_ANOMALY</code> , <code>UNKNOWN</code>) and severity (<code>INFO</code> , <code>WARNING</code> , <code>CRITICAL</code>). (c) Governed — routed through the defined IIE → GB → governance pathway.
	governance review MUST be logged, replayable, and subject to post-hoc governance review.
Enforcement	DTD classifies threats. GB routes to governance via GovernanceTicket. IIE enforces suspension or safe-mode as specified by active governance constraints.
Check Points	RUNTIME (DTD continuous monitoring)
Severity	CRITICAL (unhandled threat) FATAL (silent continuation after CRITICAL)
IIE Response	Suspend affected component + GovernanceTicket(PHYSICS_THREAT, CRITICAL)

6. Governance Architecture

6.1 Infinity Design Law — Law Hierarchy

All Infinity OS system behavior is governed by the **Infinity Design Law** hierarchy. The hierarchy comprises four levels, ordered from highest (non-overridable constitutional law) to lowest (recommended guideline). Higher-level laws constrain all lower-level laws and policies.

Level	Name	Override Policy	Violation Response
L0	Constitutional Law	Non-overrideable by construction. No governance event may override an L0 law.	Immediate HALT / fail-safe activation
L1	Statutory Law	Requires explicit L0-authorized governance event to modify.	Governance escalation + mandatory remediation
L2	Policy	Configurable within L1 bounds by authorized governance actors.	Warning + adaptive behavior enforcement
L3	Guideline	Recommended. May be deviated from with justification.	Logged for observability; no automatic enforcement

Canonical L0 Laws (Constitutional — Non-Overrideable)

Law ID	Name	Statement
L0-001	Operator Primacy	The Operator's explicit, authenticated intent supersedes all automated decisions, within legal and safety bounds. No automated system component may override a confirmed operator directive.
L0-002	Governance Supremacy	Governance constraints MUST NOT be bypassed by substrate behavior, performance concerns, runtime heuristics, or any other mechanism. This law enforces INV-4 at the constitutional level.
L0-003	Deterministic Cognition	CL behavior MUST be deterministic regardless of substrate probabilism. This law enforces INV-1 at the constitutional level and prohibits substrate nondeterminism from entering CL.

Canonical L1 Laws (Statutory)

Law ID	Name	Statement
L1-010	Epoch Lifecycle	All CL state changes MUST occur via valid, typed epoch transitions. Ad-hoc mutation is forbidden. This law enforces INV-3 at the statutory level.
L1-020	Audit Retention	Audit logs MUST be retained for a minimum configured duration and MUST be tamper-evident. Retention policy is configurable within L1 bounds. This law enforces INV-5 at the statutory level.
L1-030	PhysicsThreat Handling	CRITICAL PhysicsThreat events MUST trigger governance review and MAY suspend affected components pending decision. This law enforces INV-6 at the statutory level.

6.2 Infinity Consensus Contract

The **Infinity Consensus Contract** defines how governance decisions are proposed, validated, voted on, resolved, and enforced across all Infinity OS components. All governance activity in a CIL-conformant system **MUST** adhere to this contract.

Roles

Role	Description
Operator	Human or authorized principal with ultimate authority under L0-001. Operator decisions supersede all automated governance outcomes within legal and safety bounds.

Role	Description
Governor	The governance process or agent responsible for evaluating proposals against Design Law, managing the voting process, and enforcing decisions via IIE.
Runtime	The cognitive system subject to governance. Proposes epochs, receives governance decisions, and executes within the constraints applied by IIE.
Observer	A read-only auditor or monitoring entity. MAY read governance state and audit logs. MUST NOT participate in voting or proposal submission.

Proposal Lifecycle State Machine

DRAFT → SUBMITTED → VERIFIED → VOTING → RESOLVED → ENACTED



REJECTED



ROLLED_BACK

Articles of the Consensus Contract

Article I — Proposal Validity. Every governance proposal **MUST** include: a unique proposal ID, a declared intent, a defined scope, explicit assumptions, and a rollback plan. Proposals failing any of these requirements **MUST** be rejected at the SUBMITTED → VERIFIED transition.

Article II — Verification. Before a proposal may advance to VOTING, it **MUST** pass all three of the following checks: (a) invariant check — no active INV-* violations would result; (b) law check — no L0 or L1 violations would result; (c) impact check — performance, risk, and substrate impact are explicitly declared.

Article III — Quorum & Voting. Quorum is defined per governance domain. All votes **MUST** be cryptographically authenticated, permanently logged in the audit ledger, and individually attributable to a governance actor. Vote outcomes **MUST** be deterministic given the same authenticated vote set — no ambiguity in outcome computation is permitted.

Article IV — Conflict Resolution. Overlapping or conflicting proposals **MUST** be detected before the VOTING state. Conflicts are classified as exclusive, precedence, or mergeable. Resolution follows precedence rules: L0 > L1 > L2 > L3. Resolution rationale **MUST** be logged in the audit ledger.

Article V — Governance Over Physics. **PhysicsThreat** events **MUST** be routed through the governance pathway defined in this contract. Emergency actions taken by IIE prior to governance review **MUST** be logged, replayable, and explicitly subject to post-hoc governance review. No emergency action is permanently authorized by the fact of its execution alone.

6.3 GovernanceTicket — Canonical Governance Envelope

All governance-relevant signals — violations, physics threats, and composite events — are normalized into a single **GovernanceTicket**. This is the only format in which signals enter the governance layer.

Field	Type / Values	Description
ticket_id	UUID	Globally unique identifier for this governance event
timestamp	RFC 3339	Creation timestamp; immutable post-creation
source	IIE CL SL MSL	The originating component
kind	VIOLATION PHYSICS_THREAT COMPOSITE	Signal classification; determines payload structure
severity	INFO WARNING ERROR FATAL CRITICAL	Urgency and escalation level
subject	structured object	References: epoch_id, invariant_id, law_id, substrate_profile
payload.violation	Violation object	Populated when kind = VIOLATION or COMPOSITE
payload.physics_threat	PhysicsThreat object	Populated when kind = PHYSICS_THREAT or

Field	Type / Values	Description
		COMPOSITE
recommendation	structured object	IIE-recommended action + free-text notes
governance_state.status	OPEN UNDER_REVIEW DECIDED CLOSED	Current lifecycle state of this ticket
governance_state.decision	see below	Decision applied by governance; null until DECIDED
governance_state.decision_timestamp	RFC 3339	When the decision was issued
governance_state.decided_by	identity	Authenticated identity of the deciding governance actor

Governance Decisions (Enumeration)

Decision	Meaning
ACCEPT	The event is acknowledged; execution may continue under current conditions.
ROLLBACK	The epoch or action is rolled back; system reverts to last committed checkpoint.
WAIVE	The violation or constraint is waived for a defined scope and duration; logged and time-bounded.
AMEND_LAW	A Design Law is modified; requires L0-authorized governance event for L1 amendments.
CHANGE_SUBSTRATE	The active substrate is replaced or isolated; substrate migration is initiated.
NONE	No action taken; ticket logged for observability (typically INFO severity).

Kind Semantics: When `kind = VIOLATION`, only `payload.violation` is populated. When `kind = PHYSICS_THREAT`, only `payload.physics_threat` is populated. When `kind = COMPOSITE`, both payloads are populated — indicating that a physics anomaly directly caused an invariant breach.

7. Infinity Invariant Engine (IIE) — Component Architecture

The **Infinity Invariant Engine (IIE)** is the cross-cutting enforcement engine of CIL v1.0. It is *not* a layer — it is a runtime process that intercepts, validates, guards, verifies, and escalates across all layer transitions. IIE is always active; it cannot be suspended, paused, or bypassed by any component, including the governance layer itself.

7.1 Components

Invariant Registry (IR)

Attribute	Detail
Role	Stores and manages all active invariants and their evaluation logic. Single authoritative source of truth for the invariant set at any point in time. Immutable during active epoch execution.
Inputs	Invariant registration requests, disable requests (governance-authorized only), and query requests from PEV and other IIE components.
Outputs	Active invariant list scoped to the requesting component's evaluation domain.
Invariants Enforced	All (storage and retrieval only — IR does not evaluate invariants; evaluation is performed by the component that retrieves them).

Pre-Execution Validator (PEV)

Attribute	Detail
Role	Evaluates invariants before any CL mutation or epoch transition is authorized.

Attribute	Detail
	PEV is the gatekeeping component — no epoch may proceed to execution without PEV authorization.
Inputs	Proposed Epoch (from CL), active invariants (from IR), active governance decisions (from GB).
Outputs	ACCEPTED (epoch authorized) or REJECTED(Violation) (epoch blocked, GovernanceTicket emitted).
Invariants Enforced	INV-1 (determinism pre-check), INV-2 (boundary check), INV-3 (epoch type and legal transition), INV-4 (governance constraint check).

Runtime Guard (RG)

Attribute	Detail
Role	Enforces layer boundaries, epoch correctness, and substrate independence throughout execution. Monitors all CL↔MSL↔SL calls in-flight. Active for the full duration of epoch execution.
Inputs	Active epoch, MSL call stream, substrate telemetry anomaly signals from MSL.
Outputs	PASS (execution continues) or BLOCK(Violation) (execution halted, anomaly signal sent to DTD, GovernanceTicket emitted).
Invariants Enforced	INV-2 (boundary enforcement during execution), INV-3 (epoch semantic integrity during execution).

Post-Execution Verifier (PXV)

Attribute	Detail
Role	Verifies that resulting state, audit log entries, and governance decisions conform to invariants after execution completes. PXV is the commit gate — no epoch is committed without PXV approval.

Attribute	Detail
Inputs	Epoch result (S', outputs O, audit log entries) from MSL (EpochWorkloadResult).
Outputs	COMMIT (epoch committed to canonical state) or ROLLBACK(Violation) (epoch discarded, GovernanceTicket emitted).
Invariants Enforced	INV-1 (output hash consistency), INV-3 (idempotency verification), INV-5 (audit log completeness and tamper-evidence).

Drift & Threat Detector (DTD)

Attribute	Detail
Role	Continuously monitors substrate metrics, timing anomalies, and PhysicsThreat signals from the MSL telemetry stream. Classifies raw anomalies into typed, severity-graded PhysicsThreat objects for governance routing.
Inputs	Raw PhysicsEvent stream (via MSL), substrate metrics stream, anomaly signals from RG.
Outputs	Classified PhysicsThreat with assigned category, severity, and recommended_action — forwarded to GB for governance routing.
Invariants Enforced	INV-6 (physics threat detection and classification).

Governance Bridge (GB)

Attribute	Detail
Role	Routes violations, physics threats, and governance proposals into the Infinity Consensus Contract. Receives governance decisions and applies them to runtime behavior via IIE enforcement. GB is the exclusive interface between IIE and the governance layer.
Inputs	Violation objects (from PEV, RG, PXV), PhysicsThreat objects (from DTD), exception requests, governance decisions (via StreamTickets from governance).

Attribute	Detail
Outputs	GovernanceTicket submissions (via SubmitTicket), decision enforcement instructions to IIE runtime components.
Invariants Enforced	INV-4 (governance supremacy — ensures all decisions are applied before execution resumes), INV-6 (governance routing of classified physics threats).

7.2 IIE Integration Points

Integration Point	Direction	Operations	Protocol
CL → IIE	Outbound from CL	ProposeEpoch, QueryInvariants	Internal API
IIE → Governance	Outbound from IIE	SubmitTicket, AcknowledgeDecision	GovernanceService RPC
Governance → IIE	Inbound to IIE	StreamTickets (decision stream)	GovernanceService stream
IIE → MSL	Outbound from IIE	ExecuteEpochWorkload, StreamMetrics	MSL RPC
MSL → IIE	Inbound to IIE	ReportPhysicsThreat	Internal event bus

8. System Flows

This section defines the four canonical system flows. Each flow specifies its inputs, numbered step sequence, decision points, and outputs. All flows are normative — conformant implementations **MUST** exhibit semantically equivalent behavior.

Flow A — Epoch Proposal & Pre-Validation

Input: Proposed Epoch *E* containing intent, type, diff, and governance_context.

7. CL constructs Epoch E and calls `IIE.ProposeEpoch(E)`. Epoch transitions to state `PROPOSED`.
8. PEV retrieves the active invariant set from IR, scoped to the relevant CL domain.
9. PEV evaluates the following invariants in sequence:
 - INV-1: Determinism constraints — does the epoch introduce any substrate-dependent behavior?
 - INV-2: Boundary check — does the epoch contain any substrate-specific logic or direct SL references?
 - INV-3: Epoch validity — is the epoch type one of the five canonical types? Is the proposed state transition legal given current CL state?
 - INV-4: Governance constraint check — does the epoch violate any active governance decisions?
10. **Decision point:**
 - All checks pass → Epoch authorized. State transitions: `PROPOSED` → `AUTHORIZED`. PEV returns `EpochAuthorization`.
 - Any check fails → Epoch rejected. GB is notified. `GovernanceTicket(VIOLATION)` is emitted. State transitions: `PROPOSED` → `REJECTED`.
11. **Output:** `EpochAuthorization` or `Violation` with `GovernanceTicket`.

Flow B — Execution Guarding

Input: Authorized Epoch E (in state `AUTHORIZED`).

12. RG is activated for the scope of epoch E 's execution. Epoch transitions to state `EXECUTING`.
13. CL passes `EpochWorkload` to MSL via IIE → `MSL.ExecuteEpochWorkload()`. RG monitors all CL → MSL call traffic in-flight.

14. RG enforces during execution:

- All SL calls route through MSL only — no direct CL → SL interaction.
- No direct CL ↔ SL coupling or shared state is present.
- Substrate capabilities exercised match those declared in the SubstrateProfile (SAI Profile).

15. MSL executes the workload on SL. SL telemetry streams to DTD via `MSL.StreamMetrics()`.

16. Intermediate anomalies detected by SL stream to DTD for real-time classification.

17. **Decision point:**

- Boundary violation detected → RG issues `BLOCK(Violation)`. Execution halted. GB notified. `GovernanceTicket` emitted.
- No violation detected → Execution continues through to completion.

18. **Output:** Execution continues to PXV (Flow C) or is blocked with Violation.

Flow C — Post-Execution Verification

Input: Epoch result — resulting state S' , outputs O , and audit log entries — from MSL (`EpochWorkloadResult`).

19. PXV receives `EpochWorkloadResult`. Epoch transitions to state `VERIFYING`.

20. PXV verifies the following in sequence:

- Epoch type and state transition are internally consistent with the authorized epoch.
- `output_hash` matches the expected determinism contract (INV-1).
- Audit log entries are complete for all required event types (INV-5).
- Audit log entries are tamper-evident (INV-5).
- Key invariants are re-verified against the resulting state: INV-1 hash match, INV-5 audit completeness.

21. Decision point:

- All checks pass → IIE calls CommitEpoch. Epoch transitions: VERIFYING → COMMITTED. Canonical state is updated.
- Any check fails → ROLLBACK triggered. Canonical state is unchanged. GB notified. GovernanceTicket emitted. Epoch transitions: VERIFYING → ROLLED_BACK.

22. **Output:** EpochCommit (canonical state updated) or rollback with Violation and GovernanceTicket.

Flow D — Drift & PhysicsThreat Handling

Input: Substrate telemetry stream (via MSL.StreamMetrics), raw PhysicsEvent signals, and RG anomaly signals.

23. DTD continuously monitors metric streams from MSL.StreamMetrics() — this monitoring is always active, independent of any epoch state.

24. DTD detects an anomaly pattern in the telemetry stream (decoherence window exceeded, entanglement fidelity drop, lattice node drift, timing variance threshold breach).

25. DTD classifies the anomaly:

- Assigns category: COHERENCE_COLLAPSE | ENTANGLEMENT_POISONING | LATTICE_DRIFT | TIMING_ANOMALY | UNKNOWN
- Assigns severity: INFO | WARNING | CRITICAL

26. Decision tree based on severity:

- INFO → Log to audit ledger. No further enforcement action.
- WARNING → Log to audit ledger + notify GB. Consider throttling or substrate capacity reduction.

- CRITICAL → Notify GB immediately. Optionally trigger safe-mode, partial suspension, or substrate isolation. CL **MUST NOT** continue silently — see INV-6.
27. GB wraps the classified PhysicsThreat in a GovernanceTicket(PHYSICS_THREAT) and submits via SubmitTicket. Ticket status = OPEN.
28. Governance reviews and issues a decision: CHANGE_SUBSTRATE | ROLLBACK | ACCEPT | WAIVE.
29. IIE receives decision via StreamTickets and applies it. Calls AcknowledgeDecision when applied.
30. Governance marks GovernanceTicket status = CLOSED. Audit ledger is updated with full event chain.
31. **Output:** GovernanceTicket(CLOSED), audit log entry, applied decision.

End-to-End Canonical Example — Quantum Decoherence Leading to Invariant Breach

Scenario: A QPU substrate experiences a coherence collapse during epoch execution. The physics event propagates through the CIL enforcement chain, triggering a COMPOSITE governance event and resulting in substrate isolation and epoch rollback.

32. QPU emits raw PhysicsEvent — coherence window exceeded during epoch execution.
33. MSL normalizes the raw event → PhysicsThreat(COHERENCE_COLLAPSE, CRITICAL). Routed to DTD.
34. DTD classifies and notifies GB with the PhysicsThreat object.

35. Active epoch E — still in `EXECUTING` state — is flagged: INV-1 violation detected (deterministic output cannot be guaranteed given coherence collapse).
36. IIE constructs `GovernanceTicket(COMPOSITE, CRITICAL)` — both `payload.physics_threat` and `payload.violation` populated.
37. GB calls `SubmitTicket(ticket)`. Governance layer receives ticket with status = `OPEN`.
38. Governance reviews the `COMPOSITE` event: source = QPU, invariant = INV-1, threat = `COHERENCE_COLLAPSE`, severity = `CRITICAL`.
39. Governance issues decision: `CHANGE_SUBSTRATE`. `GovernanceTicket` updated: status = `DECIDED`, decision = `CHANGE_SUBSTRATE`.
40. IIE receives decision via `StreamTickets`.
41. IIE enforces decision: (a) isolates QPU substrate via MSL; (b) rolls back active epoch E — state transitions: `EXECUTING` → `ROLLED_BACK`; (c) system reverts to last committed checkpoint.
42. IIE calls `AcknowledgeDecision`. Governance marks `GovernanceTicket` status = `CLOSED`.
43. Audit ledger updated with complete event chain — `PhysicsEvent`, `PhysicsThreat` classification, INV-1 violation, `GovernanceTicket` lifecycle, rollback, and substrate isolation — all entries tamper-evident and replayable.
-

9. Conformance Requirements

A runtime is **CIL+IIE-conformant if and only if** all of the following requirements hold simultaneously. Partial conformance — satisfying a subset of these requirements — does not constitute conformance. Each requirement is individually verifiable and auditable.

LAYER ARCHITECTURE

- C-01** CL, MSL, and SL are implemented as distinct layers with enforced, verifiable boundaries.
- C-02** All CL↔SL traffic passes exclusively through MSL — no direct channels, shared memory, or side-band signals exist.
- C-03** CL contains no substrate-specific logic of any form.
- C-04** SL contains no cognitive decision logic and no governance constraint logic.
- C-05** MSL contains no cognitive logic and no governance logic — it is a pure translation and normalization boundary.

EPOCH SEMANTICS

- C-06** All CL state changes are expressed as typed epochs — one of: CREATE, MUTATE, ROLLBACK, GOVERNANCE, CHECKPOINT.
- C-07** Epoch transitions are deterministic — identical initial state and input always produce identical resulting state and output.
- C-08** Epochs are idempotent — replay from the same checkpoint always produces identical results.
- C-09** No ad-hoc CL state mutation occurs outside of epoch transitions — by construction, not by convention.

IIE PRESENCE

&

FUNCTION

- C-10** IIE is present and validates all epochs before execution via PEV — no epoch executes without PEV authorization.

C-11 IIE guards all execution boundaries in-flight via RG — RG is active for the full duration of all epoch executions.

C-12 IIE verifies all post-execution state and audit logs via PXV — no epoch is committed without PXV approval.

C-13 IIE detects and classifies all PhysicsThreat events via DTD — DTD monitoring is continuous and always active.

C-14 IIE integrates with a governance layer capable of vetoing or constraining CL actions via GB — GB is the exclusive IIE-governance interface.

GOVERNANCE

C-15 A governance layer is present and implements all five Articles of the Infinity Consensus Contract.

C-16 GovernanceTicket is the sole canonical envelope for all governance-relevant signals — no other signal format enters the governance layer.

C-17 Governance decisions are applied by IIE before execution resumes after any HALT or BLOCK event — no execution proceeds under an unresolved HALT.

AUDITABILITY

C-18 A tamper-evident, replayable audit ledger is maintained at all times.

C-19 All epoch transitions are logged in all states: PROPOSED, AUTHORIZED, EXECUTING, COMMITTED, REJECTED, ROLLED_BACK.

C-20 All governance decisions are logged — including decision timestamp, deciding actor, and decision rationale.

C-21 All PhysicsThreat events and all Violation events are logged at all severity levels.

C-22 Audit logs are append-only — no post-write modification or deletion is possible under any operational or maintenance procedure.

10. System Properties & Guarantees

A fully CIL+IIE-conformant system provides the following formal guarantees. These properties are emergent from the combined satisfaction of all six invariants and all 22 conformance requirements — they are not independently configured. Loss of conformance in any dimension may compromise one or more of these guarantees.

P-1 Deterministic Replay

Given any valid checkpoint and its subsequent sequence of inputs, the system can deterministically reconstruct all intermediate and final states. Reconstruction is substrate-independent — replay on any conformant substrate produces identical results.

P-2 Governance Non-Bypass

No execution path exists in a conformant system that delivers a CL state change without passing through IIE enforcement. No substrate anomaly, performance optimization, emergency override, or runtime heuristic can bypass active governance constraints. This guarantee holds even in degraded or partial-failure modes.

P-3 Substrate Portability

CIL-conformant CL behavior is semantically identical across all conformant substrate implementations. Substrates are interchangeable within their declared SubstrateProfile capability bounds without requiring CL modification. Substrate migration is a governance-authorized operation, not a CL concern.

P-4 Complete Auditability

Every system action of governance significance is present in the audit ledger. The ledger is append-only and tamper-evident — no audit entry can be retroactively modified, deleted, or suppressed by any component, operator, or maintenance procedure. The full history of all epoch transitions, governance decisions, and physics events is permanently reconstructible from the ledger alone.

P-5 Physics Threat Non-Silence

No classified CRITICAL PhysicsThreat can result in silent CL continuation. In a conformant system, every CRITICAL physics anomaly either results in a governed response (suspend, rollback, substrate change) or causes a system halt. Silent continuation after a CRITICAL event is a FATAL invariant violation under INV-6.

P-6 Epoch Atomicity

No partial epoch state is ever observable in a conformant system. An epoch is either fully committed — present in canonical state with complete audit entries — or fully absent. There is no intermediate, partial, or ambiguous epoch state in the canonical CL state. This guarantee is enforced jointly by PEV, RG, and PXV.

11. Appendix — Invariant Reference Table

The following table provides a consolidated normative reference for all six CIL v1.0 invariants. This table is a summary only — the full normative definitions are in Section 5. In the event of any discrepancy between this table and Section 5, Section 5 takes precedence.

ID	Name	Formal Constraint	Check Point(s)	Severity	IIE Response
INV-1	Deterministic Cognition	$S, I \rightarrow S', O$ identical across all conformant substrates. Nondeterminism absorbed at or below MSL.	PRE (PEV) + POST (PXV)	FATAL	HALT + Governance Ticket (COMPOSITE)
INV-2	Layer Separation	No cross-layer logic leakage. All $CL \leftrightarrow SL$ interaction exclusively via MSL. No side channels.	PRE (PEV) + RUNTIME (RG)	FATAL	HALT + Governance Ticket (VIOLATION)
INV-3	Epoch Semantics	All CL state changes via typed, deterministic, idempotent, governed epochs. No out-of-epoch mutation.	PRE (PEV) + RUNTIME (RG) + POST (PXV)	FATAL / ERROR	REJECT epoch + Governance Ticket
INV-4	Governance Supremacy	No governance bypass by substrate behavior, performance heuristic, or runtime optimization. All decisions enforced before execution resumes.	ALL (PRE + RUNTIME + POST)	FATAL	HALT + Governance Ticket (VIOLATION, CRITICAL)
INV-5	Auditability	All epoch transitions, governance decisions, PhysicsThreat events,	POST (PXV) + continuous write	FATAL / ERROR	Block epoch commit until audit entry confirmed

ID	Name	Formal Constraint	Check Point(s)	Severity	IIE Response
		violations, and IIE actions logged tamper-evidently and replayably. Logs append-only.			
INV-6	Physics Threat Handling	All substrate anomalies detected, classified, and governed. CRITICAL threats MUST NOT result in silent CL continuation .	RUNTIME (DTD — continuous)	CRITICAL / FATAL	Suspend affected component + Governance Ticket (PHYSICS_THREAT, CRITICAL)

CONFORMANCE SUMMARY

A system satisfying all requirements in Section 9 (C-01 through C-22) is CIL+IIE-conformant and thereby provides all six system property guarantees (P-1 through P-6). Conformance is binary — partial conformance is not recognized. All invariant violations require formal governance resolution under the Infinity Consensus Contract before execution may resume.